## Datenbanken – das Wichtigste für die Prüfung

Eine **Datenbank** speichert Informationen geordnet, damit man sie schnell finden, ändern und auswerten kann. Mediendesigner begegnen Datenbanken zum Beispiel bei **Websites, Online-Shops, Content-Management-Systemen (CMS), Bildarchiven oder Kundenverwaltungen**.

### 1. Tabellen, Datensätze und Felder

In einer relationalen Datenbank werden Daten meist in **Tabellen** gespeichert.

Beispiel einer Tabelle `Projekte`:

| Projekt_ID | Titel | Kunde | Status |

|---|---|---|---|

| 101 | Sommerkampagne | Nordlicht GmbH | Entwurf |

| 102 | Produktkatalog | Grün & Co. | Freigegeben |

- **Tabelle:** Sammlung zusammengehöriger Daten, hier `Projekte`
- **Datensatz/Zeile:** Angaben zu einem einzelnen Projekt
- **Feld/Spalte:** eine bestimmte Eigenschaft, etwa `Titel` oder `Status`

Ein Datensatz sollte möglichst genau **eine Sache** beschreiben – hier jeweils ein Projekt.

### 2. Primärschlüssel und Fremdschlüssel

Ein **Primärschlüssel** identifiziert jeden Datensatz eindeutig. Dafür verwendet man häufig eine ID.

Beispiel: `Projekt_ID = 101`

Ein **Fremdschlüssel** verweist auf einen Datensatz in einer anderen Tabelle. So lassen sich Tabellen miteinander verbinden.

Beispiel:

- Tabelle `Kunden`: `Kunden_ID`, `Name`, `E-Mail`
- Tabelle `Projekte`: `Projekt_ID`, `Titel`, `Kunden_ID`

Die `Kunden_ID` in `Projekte` ist ein Fremdschlüssel. Sie zeigt, welcher Kunde zu welchem Projekt gehört.

**Warum nicht den Kundennamen immer wieder in die Projekttabelle schreiben?**

Weil ein Name mehrfach vorkommen kann oder sich ändern könnte. Mit einer Kunden-ID bleiben die Daten konsistenter.

### 3. Beziehungen zwischen Tabellen

Typische Beziehungen sind:

- **1:1:** Ein Datensatz gehört genau zu einem anderen.
  Beispiel: Eine Person hat genau einen Mitarbeiterausweis.
- **1:n:** Ein Datensatz kann mit vielen Datensätzen verbunden sein.
  Beispiel: Ein Kunde hat mehrere Projekte.
- **n:m:** Viele Datensätze können mit vielen anderen verbunden sein.
  Beispiel: Ein Projekt nutzt mehrere Bilder, und ein Bild kann in mehreren Projekten vorkommen.

Eine n:m-Beziehung wird normalerweise über eine **Verknüpfungstabelle** umgesetzt, zum Beispiel `Projekt_Bilder` mit `Projekt_ID` und `Bild_ID`.

### 4. SQL: Daten abfragen und bearbeiten

**SQL** ist eine Sprache, mit der man relationale Datenbanken abfragen und verändern kann.

Alle Projekte anzeigen:

```sql
SELECT Titel, Status
FROM Projekte;
```

Nur freigegebene Projekte anzeigen:

```sql
SELECT Titel
FROM Projekte
WHERE Status = 'Freigegeben';
```

Ein neues Projekt eintragen:

```sql
INSERT INTO Projekte (Projekt_ID, Titel, Status)
VALUES (103, 'Website-Relaunch', 'Entwurf');
```

Den Status ändern:

```sql
UPDATE Projekte
SET Status = 'Freigegeben'
WHERE Projekt_ID = 103;
```

Einen Datensatz löschen:

```sql
DELETE FROM Projekte
```

WHERE Projekt_ID = 103;

```

**Wichtig:** Bei `UPDATE` und `DELETE` sollte man genau auf die `WHERE`-Bedingung achten. Ohne sie können sonst alle passenden Datensätze geändert oder gelöscht werden.

### 5. Daten filtern und sortieren

Mit `WHERE` filtert man Daten, mit `ORDER BY` sortiert man sie:

```sql
SELECT Titel, Status
FROM Projekte
WHERE Status = 'Entwurf'
ORDER BY Titel ASC;
```

`ASC` bedeutet aufsteigend, `DESC` absteigend. So könnte ein CMS beispielsweise nur Entwürfe anzeigen und alphabetisch sortieren.

### 6. Datenbankabfragen mit mehreren Tabellen

Mit einem **JOIN** verbindet man passende Datensätze aus mehreren Tabellen:

```sql
SELECT Projekte.Titel, Kunden.Name
FROM Projekte
JOIN Kunden
ON Projekte.Kunden_ID = Kunden.Kunden_ID;
```

Damit erhält man zu jedem Projekt auch den Namen des zugehörigen Kunden.
```

### 7. Normalisierung: Daten sinnvoll aufteilen

**Normalisierung** bedeutet, Daten so zu organisieren, dass Wiederholungen und Widersprüche möglichst vermieden werden.

Ungünstig wäre eine Projekttabelle, in der Kundendaten bei jedem Projekt vollständig wiederholt werden:

| Projekt | Kundenname | Kunden-E-Mail |

|---|---|---|

| Kampagne | Nordlicht GmbH | kontakt@nordlicht.de |

| Website | Nordlicht GmbH | kontakt@nordlicht.de |

Besser: Kundendaten einmal in einer `Kunden`-Tabelle speichern und über die Kunden-ID mit den Projekten verbinden.

Das reduziert doppelte Einträge und macht Änderungen sicherer: Ändert sich die E-Mail-Adresse, muss sie nur an einer Stelle aktualisiert werden.

### 8. Datenqualität und Datenschutz

Für Mediendesign-Projekte sind außerdem diese Punkte wichtig:

- **Eindeutige und einheitliche Einträge:** zum Beispiel Datumsangaben immer im gleichen Format.
- **Pflichtfelder und passende Datentypen:** Ein Datum sollte als Datum gespeichert werden, nicht als beliebiger Text.
- **Daten aktuell halten:** veraltete Texte, Bilder oder Kontaktdaten vermeiden.
- **Zugriffsrechte:** Nicht jede Person sollte alle Daten sehen oder ändern können.
- **Personenbezogene Daten schützen:** Nur notwendige Daten speichern und sie vor unbefugtem Zugriff sichern.
- **Backups:** Sicherungskopien helfen, Daten nach einem Fehler oder Ausfall wiederherzustellen.

Bei einem Bildarchiv könnten zum Beispiel Dateiname, Format, Bildgröße, Urheber, Lizenz und Schlagwörter gespeichert werden. Dabei sollte auch die Lizenz korrekt dokumentiert sein.

### Merksätze für die Prüfung

- Eine **Zeile** ist ein Datensatz, eine **Spalte** ein Feld.

- Der **Primärschlüssel** identifiziert einen Datensatz eindeutig.

- Der **Fremdschlüssel** stellt eine Verbindung zu einer anderen Tabelle her.

- Mit `SELECT` liest man Daten aus, mit `INSERT` fügt man sie ein, mit `UPDATE` ändert man sie und mit `DELETE` löscht man sie.

- Daten werden auf mehrere Tabellen verteilt, damit Wiederholungen und Fehler reduziert werden.

- Bei Änderungen und Löschungen immer prüfen, ob die `WHERE`-Bedingung stimmt.

---

Nochmal anders:

Für die Prüfung zum Mediendesigner ist es wichtig zu verstehen, dass Datenbanken nicht nur ein Thema für Informatiker sind, sondern die Grundlage für fast jedes moderne digitale Produkt (Websites, Apps, E-Commerce). Du musst nicht programmieren können wie ein Software-Entwickler, aber du musst die **Struktur** und die **Logik** dahinter verstehen, um mit Entwicklern kommunizieren zu können.

Hier ist eine Übersicht der wichtigsten Konzepte:

### 1. Grundbegriffe: Was ist eine Datenbank?

Eine Datenbank ist eine strukturierte Sammlung von Daten, die so organisiert ist, dass man sie effizient durchsuchen, ändern und verwalten kann. Im Gegensatz zu einer einfachen Excel-Liste ist eine Datenbank darauf ausgelegt, riesige Datenmengen zu bewältigen und mehrere Nutzer gleichzeitig zuzulassen.

*  **Beispiel:** Ein Online-Shop speichert nicht jede Produktseite als einzelne HTML-Datei, sondern alle Produktdaten (Preis, Bild-Pfad, Beschreibung) in einer Datenbank. Die Website „zieht“ sich diese Infos dann erst beim Aufruf.

### 2. Relationale Datenbanken (SQL)

Das ist der Standard, den du für die Prüfung kennen musst. Daten werden hier in **Tabellen** organisiert. Das Besondere ist die **Relation** (Beziehung) zwischen diesen Tabellen.

* **Tabellen, Datensätze und Felder:**
  * Eine **Tabelle** ist ein Thema (z. B. „Kunden“).
  * Ein **Datensatz** (Zeile/Row) ist ein einzelnes Objekt (z. B. „Max Mustermann“).
  * Ein **Feld** (Spalte/Column) ist eine bestimmte Eigenschaft (z. B. „E-Mail-Adresse“).
* **Primärschlüssel (Primary Key):** Jedes Objekt in einer Tabelle braucht eine eindeutige Kennung, damit es nicht verwechselt wird.
  * **Beispiel:** Eine `Kunden_ID` (z. B. ID 502). Selbst wenn es zwei Kunden namens „Max Mustermann“ gibt, unterscheiden sie sich durch ihre ID.
* **Fremdschlüssel (Foreign Key):** So werden Tabellen verknüpft. Ein Primärschlüssel aus Tabelle A wird in Tabelle B eingetragen.
  * **Beispiel:** In der Tabelle „Bestellungen“ steht nicht der Name des Kunden, sondern nur die `Kunden_ID`. So weiß das System, welcher Kunde was bestellt hat, ohne die Namen jedes Mal neu schreiben zu müssen.

### 3. Datenbank-Beziehungen (Kardinalitäten)

Du solltest wissen, wie Daten zueinander stehen:

* **1:1 (Eins-zu-eins):** Ein Datensatz in A gehört zu genau einem in B.
  * *Beispiel:* Ein Nutzer hat genau ein Benutzerprofil.
* **1:n (Eins-zu-viele):** Ein Datensatz in A kann viele Datensätze in B haben.
  * *Beispiel:* Ein Autor (1) kann viele Bücher (n) schreiben.
* **n:m (Viele-zu-viele):** Viele Datensätze in A können mit vielen in B verknüpft sein. Hierfür benötigt man in der Praxis eine „Zwischentabelle“.
  * *Beispiel:* Ein Student besucht viele Kurse, und ein Kurs hat viele Studenten.

### 4. SQL (Structured Query Language)

Du musst wahrscheinlich keine komplexen Abfragen schreiben, aber du solltest wissen, was die vier Grundoperationen (**CRUD**) bedeuten:

* **C**reate $\rightarrow$ `INSERT`: Neue Daten hinzufügen (z. B. ein neues Produkt anlegen).

*  **R**ead $\rightarrow$ `SELECT`: Daten abrufen (z. B. „Zeige mir alle Produkte unter 20 Euro“).

*  **U**pdate $\rightarrow$ `UPDATE`: Bestehende Daten ändern (z. B. einen Preis anpassen).

*  **D**elete $\rightarrow$ `DELETE`: Daten löschen (z. B. einen gelöschten Account entfernen).

### 5. Nicht-relationale Datenbanken (NoSQL)

Kurz erwähnt: Es gibt auch Datenbanken, die keine Tabellen nutzen, sondern z. B. Dokumente (wie JSON-Dateien). Diese sind flexibler und skalieren besser für riesige, unstrukturierte Datenmengen (z. B. Social-Media-Feeds).

### Zusammenfassung für den Mediendesigner

In der Prüfung wird oft gefragt, wie du ein System **planst**. Überlege dir immer:

1. Welche **Tabellen** brauche ich?
2. Was ist der **Primärschlüssel**?
3. Wie sind die Tabellen **verknüpft** (1:n oder n:m)?
4. Welche **Datentypen** kommen in die Felder (Text, Zahl, Datum, Boolean/Ja-Nein)?